

Разработка драйверов в Linux

Владислав Богданов

2026-06-16

Оглавление

- 1. Введение 1
 - 1.1. Необходимо 1
 - 1.2. Основы 1
 - 1.3. Особенности написания кода для ядра 2
 - 1.4. Инструментарий 2
 - 1.5. Системный лог 3

1. Введение

1.1. Необходимо

- Виртуальная машина
- Литература
 - Системное программирование Роберт Лав
 - Understanding the Linux Kernel Daniel P. Bovet, Marco Cesati
 - Разработка драйверов Linux Jonathan Corbet, A. Rubini

1.2. Основы

System call - обращение из кода программы к системной функции, к то му что называют системным именем system kernel symbol.

Первоначально в программировании под понятием *символ* - понималось имя функции.

То есть system call - вызов функции из ядра (пример: Open(), close(), read(), write())

Пользовательская программа - то есть я могу только пользоваться **system call**.

Системная программа - то есть я могу только пользоваться **system call** и создавать свои.

Демон - пользовательский код, так как только вызывает системные вызовы.

Написание драйверов создание новых системных вызовов. Для этого в Linux есть понятие модуля ядра (драйвер).

Связь пользовательского ПО с устройством: **device - IRQ - module - daemon - user**. Драйвером можно считать связку **IRQ - module - daemon**.

module - исходный код, текст программы, который реализует логику передачи данных от оборудования (виртуального или реального) пользователю. Модуль должен контролировать процесс подключения программы к устройству.

Виды устройств:

- Символьные устройства (90%)
- Блочные устройства (диски), отличается от символьного. Другие права доступа и блочное устройство может сообщать системе информации о геометрии диска (Цилиндры и тд).
- Сетевые интерфейсы. Для реализации сетевого интерфейса нужна сетевая железка.
- SCSI device

Взаимодействия с устройством происходит через файлы устройств.

Ключевые директории:

1. /dev
2. /proc - служебная инфо о модуле
3. /sys - можно задать параметры модулей

/proc/devices - прочитав который можно увидеть список устройств в нашей системе.

Файлы устройств создает ?

1.3. Особенности написания кода для ядра

К примеру:

1. Нет операций с плавающей точкой (так как что бы использовать floating point operations она должна быть описана на уровне языка и компилироваться под систему, в самом ядре такого типа нет, а она пишется на пользовательском уровне)
2. Используем числа с фиксированной точкой **fixed-point**



Математика написана примерно в 70-х на fortrane.

Помним

Ю

012

`Термины в ввод вывод всегда меряются относительно пользовательской программы. `
Поэтому

stdin - 0 - read() - пользовательская программа читает

stdout - 1 - write() - пользовательская программа пишет

Когда мы работаем на уровне ядра, мы пишем ответную часть для пользовательского кода поэтому в ядре все кверху ногами.

read() - модуль ядра пишет

write() - модуль ядра читает.

1.4. Инструментарий

Команды для Debian образных систем

1. sudo
2. su

3. su
4. apt update
5. apt install gcc
6. apt install build-essential
7. apt install libncurses6
8. apt install libncurses-dev
9. apt install libncursesw6

под 6 ядром

10. apt install flex
11. apt install bison
12. исходники ядра apt install linux-source под свое ядро, версия ядра *uname -r*, появятся исходники в /usr/src в архиве. Для проверки перейти в директорию и запустить make menuconfig, запустится конфигуратор ядра.

```
cd /usr/src/linux-source-x.x.x/linux-source-x.x.x
sudo make menuconfig
```

13. возможно apt upgrade

1.5. Системный лог

Системный лог находится: /var/log/. Посмотреть можно *tail -f /var/log/kern.log*. Из модуля ядра нет возможности обратиться к вводу-выводу, потому что kernel не видит стандартных терминалов. На самом деле, модуль ядра может осуществлять ввод-вывод, но механизмы кардинально отличаются от тех, к которым мы привыкли в пользовательском пространстве (user-space).

Почему в ядре нет "стандартных терминалов"?

Понятия *stdin*, *stdout*, *stderr* и функции вроде *printf()* или *scanf()* — это абстракции стандартной библиотеки C (glibc). Они работают только в пользовательском пространстве. Ядро операционной системы работает на более низком уровне:

- Оно не привязано к какому-то одному "текущему" терминалу или процессу (оно обслуживает их все).
- Оно может выполнять код до того, как пользовательское пространство и файловые системы вообще инициализированы.
- У ядра нет своего "контекста выполнения" с открытыми файловыми дескрипторами по умолчанию, как у обычной программы.

Как же тогда делать ввод-вывод из модуля ядра?

В зависимости от твоей задачи, в арсенале разработчика ядра есть несколько основных механизмов:

1. printk (Логирование и отладка)

Это прямой аналог printf для ядра. Вместо вывода на экран, printk записывает сообщения в кольцевой буфер ядра (*kernel ring buffer*). Как прочитать: Из пользовательского пространства ты можешь увидеть эти сообщения с помощью команд *dmesg* или *journalctl -k*. Пример: `_printk(KERN_INFO "My module: Hello, World!\n");`

2. Символьные устройства (Character Devices)

Это стандартный и правильный способ двустороннего обмена данными между модулем ядра и пользовательской программой. Модуль регистрирует устройство (например, `/dev/my_device`). Пользовательская программа открывает этот файл и использует обычные системные вызовы `read()`, `write()`, `ioctl()`. Ядро перехватывает эти вызовы через структуру `file_operations`, которую ты реализовал в модуле.

3. Виртуальные файловые системы: procfs (/proc) и sysfs (/sys)

Отличный способ для экспорта статуса модуля или приема простых настроек. Sysfs: Идеально для атрибутов устройств и драйверов (например, изменить значение переменной модуля через `echo 1 > /sys/class/my_class/my_device/enable`). Procfs: Исторически использовался для вывода информации о процессах и ядре, сейчас чаще используется sysfs, но *procfs* все еще применяется для специфичных задач.

4. Netlink-сокеты

Если тебе нужен сложный, асинхронный и надежный обмен сообщениями между ядром и пользовательским пространством (например, для сетевых подсистем или правил брандмауэра), используется семейство сокетов `AF_NETLINK`.

5. Прямой ввод-вывод в файлы из ядра (Advanced / Rare)

Технически ядро может открывать файлы на диске с помощью `filp_open()` и читать/писать в них через `kernel_read()` / `kernel_write()`.



Важно: Это считается плохой практикой для обычных модулей. Ядро не должно зависеть от пользовательских файловых систем для своей основной работы. Этот механизм используется только в очень специфичных случаях (например, механизмы трассировки `ftrace`, запись дампов памяти при панике ядра `kdump` или некоторые подсистемы аудита).

Резюме для обучения:

- Не пытайся "заставить" ядро писать в стандартный вывод.
- Вместо этого задай себе вопрос: "Кто и как должен получить эти данные?"
- Если это просто отладка → используй `printk`.

- Если данные должна читать пользовательская программа → создавай символическое устройство (*/dev/...*) или интерфейс в *sysfs*.